# Context-Aware LLM Routing Framework for Cloud-Deployed Applications using AI Agents

***Abstract*—As cloud applications increasingly integrate Large Language Models (LLMs), efficiently routing queries to the right model has become an important challenge. Relying on a single monolithic LLM often results in unnecessary computational overhead, higher latency, and inefficient resource usage.**

**In this work, we propose a dynamic routing framework that selects the most suitable LLM based on query intent, domain-specific performance, and system constraints. The system uses a Model Context Protocol (MCP) server connected to a benchmark database, allowing a LangGraph-based agent to compare candidate models in terms of accuracy, latency, and memory footprint before making a decision. Inspired by heterogeneous multi-agent system designs such as X-MAS, our approach combines a stateful routing graph and weighted scoring for model selection**

**On the implementation side, we use a mix of lightweight intent classification (regex-based) and LLM-assisted domain mapping, along with normalized performance metrics and asynchronous interactions with the MCP server.**

**Results suggest that dynamic, cost-aware routing can make LLM-based cloud systems more scalable and practical for real-world deployment.**

## I. Introduction and Problem Statement

The rapid proliferation of generative AI has led to the deep integration of Large Language Models (LLMs) into cloud-deployed applications ranging from conversational interfaces to complex multi-agent systems (MAS). However, this paradigm shift introduces significant operational challenges, including unpredictable and escalating cloud compute costs, variable latency under high load, and inefficient utilization of heterogeneous GPU resources.

Many existing cloud applications adopt a monolithic architecture, hardcoding a single high-parameter LLM (e.g., 70B+ models) to handle every incoming query regardless of complexity. This approach not only wastes substantial memory and computational power on trivial tasks such as simple data extraction or general-knowledge queries but also inherits uniform model biases that severely constrain collective intelligence and robust error correction in MAS environments. As highlighted in recent heterogeneous MAS research, homogeneous systems struggle with fundamental mistakes because all agents share identical weaknesses.

Furthermore, cloud providers must contend with strict GPU VRAM constraints, varying Service Level Agreements (SLAs), and the high sensitivity of LLM inference to memory footprint. Routing a lightweight query to an oversized reasoning model can inflate token costs and peak memory usage by orders of magnitude without any accuracy benefit. These issues are very common in production-scale deployments where thousands of concurrent queries must be processed efficiently.

To address these limitations, we propose a **Context-Aware LLM Routing Framework** powered by Agentic AI. A LangGraph-based routing agent intercepts every query from the upstream user query, performs intent classification, maps the query to relevant domains and subdomains, retrieves real-time performance metrics from a Model Context Protocol (MCP) server, and selects the optimal LLM using a dynamic weighted scoring algorithm. This directly translates to lower token consumption, reduced memory footprint, optimized cloud billing, and maintained high-fidelity responses.

Our key contributions are:

- Design and implementation of a fully stateful LangGraph workflow integrating MCP tools for real-time benchmarking.
- Novel intent classification, domain-subdomain mapping, and weighted scoring with metric normalization tailored to cloud constraints.
- Comprehensive empirical evaluation across multiple domains showing significant resource savings.
- Open-source-ready architecture that builds directly on X-MAS-Bench and MCP standards for heterogeneous MAS.

The remainder of this paper is organized as follows: Section II reviews core technologies. Section III surveys related work. Section IV details the proposed methodology and full implementation. Finally, Sections V and VI discuss limitations and future directions.

## II. Background and Core Technologies

### A. AI Agents and Agentic Workflows

AI agents are autonomous software entities capable of perceiving their environment, reasoning over context, executing tool calls, and maintaining persistent memory to accomplish complex objectives. In contrast to traditional rule-based scripts, agentic workflows incorporate iterative cycles of perception, planning, action, and reflection.

In our framework, the central routing agent augments a base LLM with structured state management and tool integration. The complete state is captured in the `AgentState` TypedDict (see Listing 1), which tracks query intent, available models, domains, metrics, and the final selected LLM.

```
class AgentState(TypedDict):
    messages: list
    step: str
    query: str
    intent: str
    models: list
    domains: list
    subdomains: dict
```

```
    context: str
    metrics: dict
    selected_llm: Any
```

Listing 1: AgentState

### B. Agentic Routing with LangGraph

While LangChain enables basic tool-calling agents, complex stateful orchestration requires LangGraph. LangGraph models the entire workflow as a directed graph where nodes represent individual steps (e.g., initialization, metric collection) and edges define control flow, including conditional branching and cycles.

Our graph construction (Listing 2) consists of three core nodes: `init`, `collect_metrics`, and `respond`. The use of `MemorySaver` enables checkpointing for reliable recovery in cloud environments.

```
def _build_graph(self, cp):
    g = StateGraph(AgentState)
    g.add_node("init", self.init_step)
    g.add_node("collect_metrics",
    self.collect_metrics_step)
    g.add_node("respond", self.respond_step)
    g.add_edge("init", "collect_metrics")
    g.add_edge("collect_metrics", "respond")
    g.set_entry_point("init")
    return g.compile(checkpointer=cp)
```

Listing 2: LangGraph construction

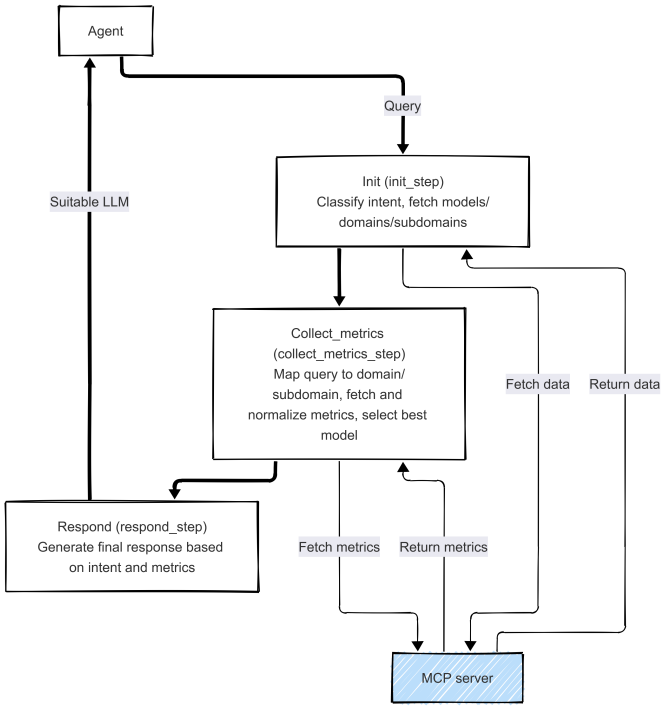


Fig. 1: Detailed schematic of the AI routing agent showing interaction with MCP tools and persistent memory (from implementation).

### C. Model Context Protocol (MCP) for Benchmarking

The Model Context Protocol (MCP) provides a standardized client-server interface for LLMs to access external tools and structured benchmarks. Our routing application acts as the MCP client, while a FastMCP server (launched via `MultiServerMCPClient`) hosts the performance score matrix.

Available tools include `list_models`, `list_domains`, `list_sub_domains`, `get_metric_domain`, and `get_metric_domain_subdomain`. This architecture enables real-time, domain-aware metric retrieval without hardcoding benchmarks.

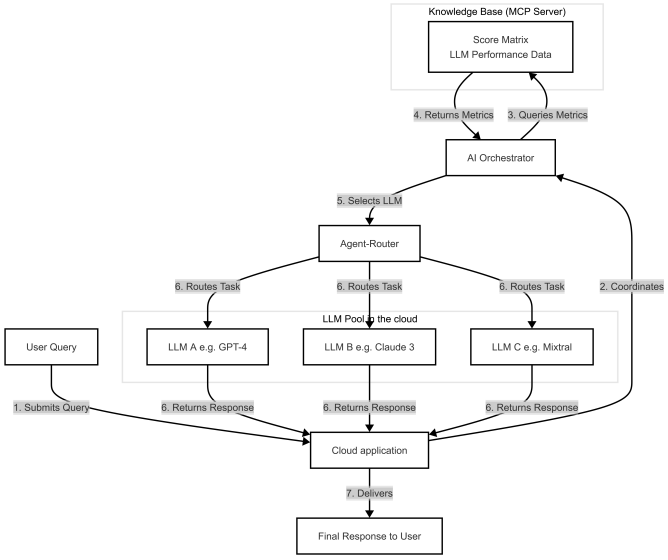


Fig. 2: End-to-end framework workflow: User query interception → LangGraph routing → MCP metric fetch → optimal LLM dispatch.

## III. Related Work

### A. Heterogeneous Multi-Agent Systems

Traditional LLM-based multi-agent systems (MAS) often rely on homogeneous architectures where all agents are powered by the same underlying LLM. While this simplifies implementation, it limits overall performance due to shared model biases, uniform error patterns, and the inability to leverage complementary strengths across different models.

Recent advancements address this limitation through heterogeneous LLM-driven MAS. The seminal work on X-MAS [1] introduces the paradigm of heterogeneous LLM-driven multi-agent systems (X-MAS), where individual agents are powered by diverse LLMs instead of a single model. This approach elevates system performance by harnessing the collective intelligence of specialized models. The authors present X-MAS-Bench, a comprehensive evaluation suite that assesses 28 different LLMs across five domains (with 21 test sets) and five MAS-related functions, conducting over 1.7 million evaluations. Their findings reveal that no single LLM

excels universally; instead, optimal model assignments vary significantly by domain and function.

Empirically, transitioning from homogeneous to heterogeneous configurations yields substantial gains without requiring architectural redesign. In chatbot-only MAS scenarios, heterogeneous setups achieve up to 8.4% performance improvement on the MATH dataset. These results underscore the transformative potential of heterogeneity in overcoming the fundamental weaknesses of monolithic MAS, directly motivating our dynamic routing framework that automatically assigns the most suitable LLM to each agent or query based on real-time context and metrics.

### B. LLM Routing and Cascading Techniques

As the number of available LLMs grows, with varying capabilities, costs, and latencies, static assignment becomes inefficient. Dynamic LLM routing and cascading have emerged as effective strategies to optimize the accuracy-cost-latency trade-off.

A recent comprehensive survey on dynamic model routing and cascading for efficient LLM inference [4] categorizes approaches into multiple paradigms, including difficulty-aware routing, human-preference alignment, clustering-based methods, uncertainty quantification, reinforcement learning, and multimodality-aware routing. Unlike Mixture-of-Experts (MoE) architectures that perform routing within a single model, these techniques route queries across independently trained LLMs at inference time. The survey emphasizes that practical routing systems are often compositional, combining multiple paradigms under operational constraints such as latency budgets and cloud resource limits.

Complementary to general routing, cascading techniques start with smaller or faster models for initial processing and escalate to more powerful (and expensive) models only when necessary. This "do more with less" philosophy has been shown to deliver near-premium quality at significantly reduced cost.

Our context-aware routing framework aligns closely with these directions by combining difficulty/domain-aware selection (via intent classification and domain-subdomain mapping) with metric-driven weighted scoring, while supporting potential future extensions to cascading and language-conditioned constraints.

### C. Model Context Protocol (MCP) and Agent Communication

The Model Context Protocol (MCP), introduced as a standardized interface for connecting LLMs to external tools and data sources, has rapidly gained traction as the "USB-C of AI." It addresses the fragmentation of bespoke tool integrations and enables seamless agent-tool interactions in real-world deployments.

To rigorously evaluate LLMs in MCP environments, Luo et al. [2] introduced MCP-Universe, the first comprehensive benchmark designed for realistic, hard tasks involving interaction with actual MCP servers. The benchmark spans six domains and stresses key challenges such as long-context management (as token counts grow rapidly with interaction steps) and the "unknown-tools" problem, where agents must discover and correctly use unfamiliar MCP servers. Even state-of-the-art models like GPT-5, Grok-4, and Claude-4.0-Sonnet show significant limitations, with success rates ranging from 29% to 44%. The benchmark also reveals that enterprise-level agents (e.g., Cursor) do not consistently outperform standard ReAct-style frameworks in live MCP settings.

Overall, while existing works establish the value of heterogeneity, routing, and standardized protocols separately, our contribution integrates them into a unified, agentic routing system tailored for cost-aware cloud applications.

### D. Cloud LLM Serving and Resource Optimization

Complementary to infrastructure-level optimizations, query-level **LLM routing** has emerged as a powerful technique for cost reduction without sacrificing response quality. Dynamic routing systems analyze incoming queries and direct them to the most appropriate model, routing simple or routine tasks to smaller, faster, and cheaper models while reserving powerful (and expensive) models for complex reasoning or domain-specific tasks. Production deployments and benchmarks report cost savings ranging from 30% to 85%, with many systems achieving 40–70% reductions in token costs while maintaining 90%+ of the quality of premium models.

Our proposed context-aware LLM routing framework directly addresses these challenges at the application layer. By intercepting queries from cloud-deployed multi-agent systems/applications and employing a LangGraph-based agent, the framework performs accurate intent classification, domain-subdomain mapping, and real-time retrieval of performance metrics via the Model Context Protocol (MCP) server. The weighted scoring algorithm then selects the most suitable LLM based on normalized accuracy, latency, and memory metrics, with dynamically adjustable weights tailored to deployment priorities (e.g., latency-sensitive vs. accuracy-critical workloads).

This precise and context-aware LLM selection enables the framework to consistently bypass oversized, high-cost models for simpler queries, resulting in substantial reductions in token usage, peak memory consumption (RSS), and overall inference latency. Since cloud billing is predominantly driven by token volume and GPU-hour usage, these optimizations translate directly into lower operational costs and improved resource efficiency.

By bridging query-level intelligence with infrastructure-level heterogeneity awareness, our approach complements lower-level serving systems and provides a practical, agentic layer for production MAS deployments or AI cloud applications.

## IV. Methodology and Implementation

### A. Performance and Cost Metrics

Metrics retrieved via MCP include latency (ms), peak memory (MB), token counts, throughput (tokens/sec), and domain-specific accuracy.

### B. Intent Classification and Domain Mapping

Intent is classified using regex patterns followed by LLM-assisted mapping to domains/subdomains.

### C. Weighted Scoring Algorithm

The core selection logic normalizes metrics and computes a weighted score (Equation 1). Default weights prioritize accuracy (0.7) while penalizing latency and memory.

$$S = w_{\text{acc}} \cdot \text{Accuracy} + w_{\text{lat}} \cdot \text{Latency} + w_{\text{mem}} \cdot \text{Memory} \quad (1)$$

```
def normalize_accuracy(self, v):
    return v / 100 if isinstance(v, (int, float)) and v > 1 else v

def normalize_metric(self, value, metric_type, all_values):
    """
    Applies min-max normalization.
    Inverts values for cost-based metrics like latency and memory.
    """
    # implementation here
    ...

def select_best_llm(self, metrics, key):
    """
    Computes weighted scores and selects the best model.
    """
    # implementation here
    ...
```

Listing 3: Normalization and model selection logic

## V. Discussion and Limitations

While effective, the framework incurs minor MCP query overhead (mitigated by caching) and requires periodic re-benchmarking as new LLMs emerge. Multi-domain queries are handled via parallel tool calls but may increase latency in edge cases.

## VI. Conclusion and Future Work

We presented a production-ready, context-aware LLM routing framework that operationalizes X-MAS and MCP principles for cloud environments. The LangGraph + weighted scoring design delivers substantial resource efficiency gains.

Future directions include:

- Integration with emerging LLMs and expanded X-MAS-Bench sub-domains.
- Reinforcement-learning-based adaptive weight tuning.
- Kubernetes-native deployment with auto-scaling and vLLM serving.
- Edge-cloud hybrid routing for low-latency applications.
- Broader real-world evaluation on enterprise MAS workloads.

This work establishes a foundation for sustainable, intelligent AI cloud applications.

## References


[1] R. Ye et al., "X-MAS: Towards Building Multi-Agent Systems with Heterogeneous LLMs," arXiv:2505.16997, 2025.
[2] Z. Luo et al., "MCP Universe: Evaluating Model Context Protocol Servers for Agent-Based Systems," arXiv:2508.14704, 2025.
[3] Du et al., "Improving Factuality and Reasoning in Language Models through Multiagent Debate," arXiv:2401.06322, 2024.
[4] J. Dekoninck et al., "Dynamic Model Routing and Cascading for Efficient LLM Inference: A Survey," arXiv:2603.04445, 2026.
[5] "LLM Routing as Reasoning: A MaxSAT View," arXiv:2603.13612, 2026.
[6] LangChain Inc., "LangGraph: Build Resilient Language Agents as Graphs," 2024.
[7] K. Jain et al., "Serving Models, Fast and Slow: Optimizing Heterogeneous LLM Inferencing," Microsoft Research, 2025.
[8] H. Tran et al., "Multi-Agent Collaboration Mechanisms: A Survey of LLMs," arXiv:2501.06322, 2025.
[9] "Evaluating large language model agents for automation," Nature Communications, 2025.
[10] A. Sarkar et al., "Survey of LLM Agent Communication with MCP," arXiv:2506.05364, 2025.
[11] "A Survey of Context Engineering for Large Language Models," arXiv:2507.13334, 2025.
[12] J. Dekoninck et al., "A Unified Approach to Routing and Cascading for LLMs," arXiv:2410.10347, 2024 (v3 2025).
[13] Microsoft Research, "AutoGen: Enabling Next-Gen LLM Applications," 2025.
[14] "MetaGPT: Meta Programming for Multi-Agent Collaborative Framework," 2025.
[15] Various, "Model-Aware Scheduling for LLM Inference in Cloud Environments," 2025.
[16] "Heterogeneous Cluster Orchestration with OpenStack for LLM Serving," 2025.
[17] "Human-in-the-Loop Oversight in Agentic LLM Systems," arXiv preprint, 2025.
[18] Y. Zhang et al., "Cauchy: A Cost-Efficient LLM Serving System through Adaptive Heterogeneous Deployment," in Proc. ACM Symposium on Cloud Computing (SoCC), 2025.
[19] Y. Jiang et al., "Demystifying Cost-Efficiency in LLM Serving over Heterogeneous GPUs," arXiv:2502.00722, 2025.
[20] Anyscale Team, "Building an LLM Router for High-Quality and Cost-Effective Responses," 2024 (updated 2025).
[21] IBM Research, "LLM Routing for Quality, Low-Cost Responses," 2024.
[22] NVIDIA, "Deploying the NVIDIA AI Blueprint for Cost-Efficient LLM Routing," 2025.
[23] "X-MAS-Bench: Comprehensive Evaluation Suite," 2025.
[24] Fireworks AI Documentation, 2025.